

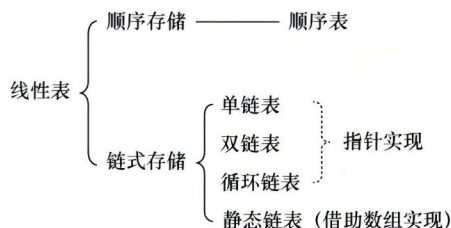
第 2 章

线 性 表

【考纲内容】

- (一) 线性表的基本概念
- (二) 线性表的实现
顺序存储；链式存储
- (三) 线性表的应用

【知识框架】



扫一扫



视频讲解

【复习提示】

线性表是算法题命题的重点。应牢固掌握其在两种存储结构（顺序存储与链式存储）下的各种基本操作，并在平时的学习中注重动手能力的培养。另需提醒的是，算法最重要的是思想！考场时间紧迫，在试卷上并不要求代码具备实际可执行性，应着重清晰地表达出算法的核心思想与关键步骤，而不必过于拘泥于语法、边界等细节。此外，即使采用暴力法，通常也能获得大部分分数，因此在时间紧张时建议直接采用。注意，算法题只能使用 C/C++ 语言实现。

2.1 线性表的定义和基本操作

2.1.1 线性表的定义

线性表是由 n ($n \geq 0$) 个相同数据类型的元素组成的有限序列。其中 n 为表长；当 $n = 0$ 时，线性表为空表。若用 L 表示一个线性表，则其一般形式为

$$L = (a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n)$$

式中， a_1 是唯一的“第一个”数据元素，也称表头元素； a_n 是唯一的“最后一个”数据元素，也称表尾元素。除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继（“直接前驱”与“前驱”、“直接后继”与“后继”通常被视为同义词）。以上即为线性表的逻辑特性，这种一对一的线性逻辑关系正是线性表名称的由来。

由此，可总结出线性表的特点如下：

- 表中元素的个数有限。

- 表中元素具有逻辑上的顺序性,即存在明确的先后次序。
- 表中元素均为数据元素,每个元素在线性表中被视为一个不可分割的单位。
- 表中元素的数据类型都相同,这意味着每个元素占有相同大小的存储空间。
- 表中元素具有抽象性,即仅讨论元素之间的逻辑关系,而不考虑其究竟表示什么内容。

注意

线性表是一种逻辑结构,表示元素之间一对一的相邻关系。顺序表和链表是两种存储结构,二者属于不同层面的概念,因此不要将其混淆。

2.1.2 线性表的基本操作

一个数据结构的基本操作是指其最核心、最基本的操作集合,其他较复杂的功能通常可通过调用这些基本操作来实现。线性表的主要操作如下。

- `InitList(&L)`: 初始化表。构造一个空的线性表。
- `Length(L)`: 求表长。返回线性表 L 的长度,即其中数据元素的个数。
- `LocateElem(L,e)`: 按值查找操作。在表 L 中查找值为 e 的元素,并返回其位置。
- `GetElem(L,i)`: 按位查找操作。获取表 L 中第 i 个位置的元素值。
- `ListInsert(&L,i,e)`: 插入操作。在表 L 的第 i 个位置插入指定元素 e 。
- `ListDelete(&L,i,&e)`: 删除操作。删除表 L 中第 i 个位置的元素,并用 e 返回被删元素的值。
- `PrintList(L)`: 输出操作。按前后顺序输出线性表 L 的所有元素值。
- `Empty(L)`: 判空操作。若 L 为空表,则返回 `true`,否则返回 `false`。
- `DestroyList(&L)`: 销毁操作。销毁线性表,并释放其占用的内存空间。

注意

① 基本操作的具体实现取决于所用的存储结构,存储结构不同,算法的实现方式也不同。② 符号 `&` 表示 C++ 语言中的引用调用;在 C 语言中,可通过指针实现相同效果。

2.1.3 本节试题精选

单项选择题

01. 线性表是具有 n 个 () 的有限序列。
A. 数据表 B. 字符 C. 数据元素 D. 数据项
02. 下列几种描述中, () 是一个线性表。
A. 由 n 个实数组成的集合 B. 由 100 个字符组成的序列
C. 所有整数组成的序列 D. 邻接表
03. 在线性表中,除开始元素外,每个元素 ()。
A. 只有唯一的前驱元素 B. 只有唯一的后继元素
C. 有多个前驱元素 D. 有多个后继元素
04. 若非空线性表中的元素既没有直接前驱,又没有直接后继,则该表中有 () 个元素。
A. 1 B. 2 C. 3 D. n

2.1.4 答案与解析

单项选择题

01. C

线性表是由具有相同数据类型的有限数据元素组成的,数据元素是由数据项组成的。

02. B

线性表定义的要求为：相同数据类型、有限序列。选项 C 的元素个数是无穷个，错误；选项 A 集合中的元素没有前后驱关系，错误；选项 D 属于一种存储结构，本题要求选出的是一个具体的线性表，不要将二者混为一谈。只有选项 B 符合线性表定义的要求。

03. A

线性表中，除最后一个（或第一个）元素外，每个元素都只有一个后继（或前驱）元素。

04. A

线性表中的第一个元素没有直接前驱，最后一个元素没有直接后继；当线性表中仅有一个元素时，该元素既没有直接前驱，又没有直接后继。

2.2 线性表的顺序表示

2.2.1 顺序表的定义

考点追踪 ▶▶（算法题）顺序表的应用（2010、2011、2018、2020、2025）

线性表的顺序存储也称顺序表。它是用一组地址连续的存储单元依次存储线性表中的数据元素，使得逻辑上相邻的元素在物理位置上也相邻。第 1 个元素存储在顺序表的起始位置，第 i 个元素的存储位置之后紧接着存放的是第 $i+1$ 个元素，称 i 为元素 a_i 在顺序表中的位序。因此，顺序表的特点是：表中元素的逻辑顺序与其物理存储顺序完全一致。

设顺序表 L 的起始存储地址为 $LOC(A)$ ，每个数据元素占用的存储空间大小为 $sizeof(ElemType)$ ，则顺序表的存储结构如图 2.1 所示。

数组下标	顺序表	内存地址
0	a_1	$LOC(A)$
1	a_2	$LOC(A) + sizeof(ElemType)$
	$\vdots$	
$i-1$	a_i	$LOC(A) + (i-1) \times sizeof(ElemType)$
	$\vdots$	
$n-1$	a_n	$LOC(A) + (n-1) \times sizeof(ElemType)$
	$\vdots$	
$MaxSize-1$	$\vdots$	$LOC(A) + (MaxSize-1) \times sizeof(ElemType)$

图 2.1 线性表的顺序存储结构

由于任意数据元素的存储地址与顺序表起始地址之间的偏移量与其位序呈线性关系，因此可在 $O(1)$ 时间内直接访问表中任意位置的元素，这种特性使得线性表的顺序存储结构属于随机存取。在高级程序设计语言中，通常使用数组来实现顺序表。

注 意

线性表中元素的位序从 1 开始，而数组中元素的下标从 0 开始，注意二者之间的转换。

假定线性表的元素类型为 `ElemType`，则静态分配的顺序表存储结构描述为

```
#define MaxSize 50           //定义线性表的最大长度
typedef struct{
    ElemType data[MaxSize];    //顺序表的元素
```

```

    L.length=0;           //顺序表初始长度为0
    L.MaxSize=InitSize;   //初始存储容量
}

```

其中，MaxSize 表示当前分配的存储空间上限。当插入元素导致空间不足时，需要扩容。

2. 插入操作

在顺序表 L 的第 i ($1 \leq i \leq L.length+1$) 个位置插入新元素 e 。若 i 超出合法范围，或存储空间已满，则插入失败，返回 false；否则，将第 i 个元素及其后的所有元素依次后移一位，腾出一个空位置插入 e ，表长加 1，返回 true。

```

bool ListInsert(SqList &L, int i, ElemType e) {
    if (i < 1 || i > L.length + 1)           //判断 i 的范围是否有效
        return false;
    if (L.length >= MaxSize)                 //当前存储空间已满
        return false;
    for (int j = L.length; j >= i; j--)       //将第 i 个元素及之后的元素后移
        L.data[j] = L.data[j - 1];
    L.data[i - 1] = e;                       //在位置 i 插入 e (注意下标转换)
    L.length++;                             //表长加 1
    return true;
}

```

注意

区分位序 (从 1 开始) 与数组下标 (从 0 开始)。为什么判断插入位置时使用 $length+1$ ，而移动元素的 for 循环中使用 $length$ ？因为合法插入位置包括第 $n+1$ 位，但移动元素时最多从第 n 位开始后移。

最好情况：在表尾插入 ($i = n + 1$)，无须移动元素，时间复杂度为 $O(1)$ 。

最坏情况：在表头插入 ($i = 1$)，需移动全部 n 个元素，时间复杂度为 $O(n)$ 。

平均情况：设在第 i 个位置插入的概率为 $p_i = 1/(n + 1)$ ，则平均移动次数为

$$\sum_{i=1}^{n+1} p_i (n - i + 1) = \sum_{i=1}^{n+1} \frac{1}{n+1} (n - i + 1) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{1}{n+1} \frac{n(n+1)}{2} = \frac{n}{2}$$

因此，插入操作的平均时间复杂度为 $O(n)$ 。

3. 删除操作

删除顺序表 L 中第 i ($1 \leq i \leq L.length$) 个位置的元素，并通过引用参数 e 返回其值。若 i 非法，返回 false；否则，保存被删元素，将其后所有元素前移一位，表长减 1，返回 true。

```

bool ListDelete(SqList &L, int i, ElemType &e) {
    if (i < 1 || i > L.length)           //判断 i 的范围是否有效
        return false;
    e = L.data[i - 1];                   //将被删除的元素赋给 e
    for (int j = i; j < L.length; j++)    //将第 i 个位置后的元素前移
        L.data[j - 1] = L.data[j];
    L.length--;                          //表长减 1
    return true;
}

```

最好情况：删除表尾元素 ($i = n$)，无须移动元素，时间复杂度为 $O(1)$ 。

最坏情况：删除表头元素 ($i = 1$)，需移动其余 $n - 1$ 个元素，时间复杂度为 $O(n)$ 。

平均情况：设删除第 i 个元素的概率为 $p_i = 1/n$ ，则平均移动次数为

$$\sum_{i=1}^n p_i (n - i) = \sum_{i=1}^n \frac{1}{n} (n - i) = \frac{1}{n} \sum_{i=1}^n (n - i) = \frac{1}{n} \frac{n(n-1)}{2} = \frac{n-1}{2}$$

因此,删除操作的平均时间复杂度为 $O(n)$ 。

可见,顺序表的插入和删除操作的时间开销主要耗费在移动元素上,而移动元素的个数取决于操作位置。图 2.2 显示了顺序表插入和删除操作前、后的状态,以及数据元素在存储空间中的位置和表长变化。图 2.2(a)将第 4~7 个元素从后往前依次后移一位,为新元素腾出位置。图 2.2(b)将第 5~7 个元素从前往后依次前移一位,填补删除后的空缺。

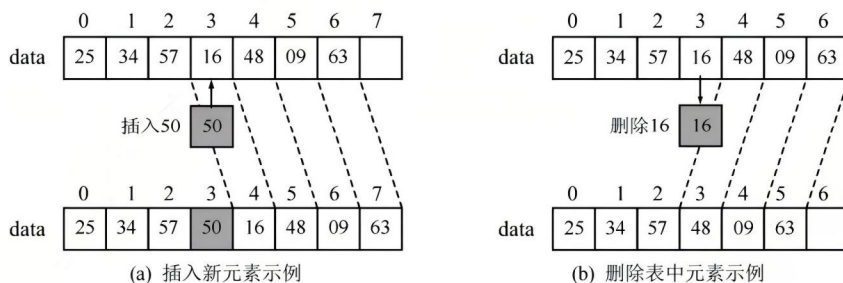


图 2.2 顺序表的插入和删除

4. 按值查找 (顺序查找)

在顺序表 L 中查找第一个值等于 e 的元素, 返回其位序; 若未找到, 则返回 0。

```
int LocateElem(SqList L, ElemType e){
    int i;
    for(i=0; i<L.length; i++){
        if(L.data[i]==e)
            return i+1;        //下标为 i 的元素值等于 e, 返回其位序 i+1
    }
    return 0;                  //查找失败
}
```

最好情况: 目标元素在表头 ($i=1$), 比较 1 次, 时间复杂度为 $O(1)$ 。

最坏情况: 目标元素在表尾或不存在, 需要比较 n 次, 时间复杂度为 $O(n)$ 。

平均情况: 设目标元素在第 i 位的概率为 $p_i = 1/n$, 则平均比较次数为

$$\sum_{i=1}^n p_i \cdot i = \sum_{i=1}^n \frac{1}{n} \cdot i = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2}$$

因此, 按值查找的平均时间复杂度为 $O(n)$ 。

顺序表的按序号查找非常简单, 直接通过数组下标访问即可, 时间复杂度为 $O(1)$ 。

2.2.3 本节试题精选

一、单项选择题

01. 下列叙述中, () 是顺序存储结构的优点。

- A. 存储密度大
- B. 插入运算方便
- C. 删除运算方便
- D. 方便地运用于各种逻辑结构的存储表示

02. 下列关于顺序表的叙述中, 正确的是 ()。

- A. 顺序表可以利用一维数组表示, 因此顺序表与一维数组在逻辑结构上是相同的
- B. 在顺序表中, 逻辑上相邻的元素物理位置上不一定相邻
- C. 顺序表和一维数组一样, 都可以进行随机存取
- D. 在顺序表中, 每个元素的类型不必相同

03. 通常说顺序表具有随机存取的特性, 指的是 ()。

- A. 查找值为 x 的元素的时间与顺序表中元素个数 n 无关
B. 查找值为 x 的元素的时间与顺序表中元素个数 n 有关
C. 查找序号为 i 的元素的时间与顺序表中元素个数 n 无关
D. 查找序号为 i 的元素的时间与顺序表中元素个数 n 有关
04. 一个顺序表所占用的存储空间大小与 () 无关。
A. 表的长度
B. 元素的存放顺序
C. 元素的类型
D. 元素中各字段的类型
05. 若线性表最常用的操作是存取第 i 个元素及其前驱和后继元素的值, 为了提高效率, 应采用 () 的存储方式。
A. 单链表
B. 双链表
C. 循环单链表
D. 顺序表
06. 一个线性表最常用的操作是存取任意一个指定序号的元素并在最后进行插入、删除操作, 则利用 () 存储方式可以节省时间。
A. 顺序表
B. 双链表
C. 带头结点的循环双链表
D. 循环单链表
07. 在 n 个元素的线性表的数组表示中, 时间复杂度为 $O(1)$ 的操作是 ()。
I. 访问第 i ($1 \leq i \leq n$) 个结点和求第 i ($2 \leq i \leq n$) 个结点的直接前驱
II. 在最后一个结点后插入一个新的结点
III. 删除第 1 个结点
IV. 在第 i ($1 \leq i \leq n$) 个结点后插入一个结点
A. I
B. II、III
C. I、II
D. I、II、III
08. 设线性表有 n 个元素, 严格说来, 以下操作中, () 在顺序表上实现要比在链表上实现的效率高。
I. 输出第 i ($1 \leq i \leq n$) 个元素值
II. 交换第 3 个元素与第 4 个元素的值
III. 顺序输出这 n 个元素的值
A. I
B. I、III
C. I、II
D. II、III
09. 在一个长度为 n 的顺序表中删除第 i ($1 \leq i \leq n$) 个元素时, 需向前移动 () 个元素。
A. n
B. $i-1$
C. $n-i$
D. $n-i+1$
10. 对于顺序表, 访问第 i 个位置的元素和在第 i 个位置插入一个元素的时间复杂度为 ()。
A. $O(n), O(n)$
B. $O(n), O(1)$
C. $O(1), O(n)$
D. $O(1), O(1)$
11. 对于顺序存储的线性表, 其算法时间复杂度为 $O(1)$ 的运算应该是 ()。
A. 将 n 个元素从小到大排序
B. 删除第 i ($1 \leq i \leq n$) 个元素
C. 改变第 i ($1 \leq i \leq n$) 个元素的值
D. 在第 i ($1 \leq i \leq n$) 个元素后插入一个新元素
12. 顺序表的插入算法中, 当 n 个空间已满时, 可再申请增加分配 m 个空间, 若申请失败, 则说明系统没有 () 可分配的存储空间。
A. m 个
B. m 个连续
C. $n+m$ 个
D. $n+m$ 个连续
13. 【2023 统考真题】在下列对顺序存储的有序表 (长度为 n) 实现给定操作的算法中, 平均时间复杂度为 $O(1)$ 的是 ()。

- A. 查找包含指定值元素的算法 B. 插入包含指定值元素的算法
C. 删除第 i ($1 \leq i \leq n$) 个元素的算法 D. 获取第 i ($1 \leq i \leq n$) 个元素的算法

二、综合应用题

- 从顺序表中删除具有最小值的元素（假设唯一）并由函数返回被删元素的值。空出的位置由最后一个元素填补，若顺序表为空，则显示出错信息并退出运行。
- 设计一个高效算法，将顺序表 L 的所有元素逆置，要求算法的空间复杂度为 $O(1)$ 。
- 对长度为 n 的顺序表 L ，编写一个时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法，该算法删除顺序表中所有值为 x 的数据元素。
- 从顺序表中删除其值在给定值 s 和 t 之间（包含 s 和 t ，要求 $s < t$ ）的所有元素，若 s 或 t 不合理或顺序表为空，则显示出错信息并退出运行。
- 从有序顺序表中删除所有其值重复的元素，使表中所有元素的值均不同。
- 将两个有序顺序表合并为一个新的有序顺序表，并由函数返回结果顺序表。
- 已知在一维数组 $A[m+n]$ 中依次存放两个线性表 $(a_1, a_2, a_3, \dots, a_m)$ 和 $(b_1, b_2, b_3, \dots, b_n)$ 。编写一个函数，将数组中两个顺序表的位置互换，即将 $(b_1, b_2, b_3, \dots, b_n)$ 放在 $(a_1, a_2, a_3, \dots, a_m)$ 的前面。
- 线性表 $(a_1, a_2, a_3, \dots, a_n)$ 中的元素递增有序且按顺序存储于计算机内。要求设计一个算法，完成用最少时间在表中查找数值为 x 的元素，若找到，则将其与后继元素位置相交换，若找不到，则将其插入表中并使表中元素仍然递增有序。
- 给定三个序列 A 、 B 、 C ，长度均为 n ，且均为无重复元素的递增序列，请设计一个时间上尽可能高效的算法，逐行输出同时存在于这三个序列中的所有元素。例如，数组 A 为 $\{1, 2, 3\}$ ，数组 B 为 $\{2, 3, 4\}$ ，数组 C 为 $\{-1, 0, 2\}$ ，则输出 2。要求：
 - 给出算法的基本设计思想。
 - 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
 - 说明你的算法的时间复杂度和空间复杂度。
- 【2010 统考真题】设将 n ($n > 1$) 个整数存放于一维数组 R 中。设计一个在时间和空间两方面都尽可能高效的算法。将 R 中保存的序列循环左移 p ($0 < p < n$) 个位置，即将 R 中的数据由 $(X_0, X_1, \dots, X_{n-1})$ 变换为 $(X_p, X_{p+1}, \dots, X_{n-1}, X_0, X_1, \dots, X_{p-1})$ 。要求：
 - 给出算法的基本设计思想。
 - 根据设计思想，采用 C 或 C++ 或 Java 语言描述算法，关键之处给出注释。
 - 说明你所设计算法的时间复杂度和空间复杂度。
- 【2011 统考真题】一个长度为 L ($L \geq 1$) 的升序序列 S ，处在第 $\lceil L/2 \rceil$ 个位置的数称为 S 的中位数。例如，若序列 $S_1 = (11, 13, 15, 17, 19)$ ，则 S_1 的中位数是 15，两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2 = (2, 4, 6, 8, 20)$ ，则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B ，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：
 - 给出算法的基本设计思想。
 - 根据设计思想，采用 C 或 C++ 或 Java 语言描述算法，关键之处给出注释。
 - 说明你所设计算法的时间复杂度和空间复杂度。
- 【2013 统考真题】已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$ ，其中 $0 \leq a_i < n$ ($0 \leq i < n$)。若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2$ ($0 \leq p_k < n, 1 \leq k \leq m$)，则称 x 为 A 的主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$ ，则 5 为主元素；又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$ ，则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中，请设计一个尽可能高效的算法，

找出 A 的主元素。若存在主元素，则输出该元素；否则输出 -1。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 或 Java 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

13. 【2018 统考真题】给定一个含 n ($n \geq 1$) 个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如，数组 $\{-5, 3, 2, 3\}$ 中未出现的最小正整数是 1；数组 $\{1, 2, 3\}$ 中未出现的最小正整数是 4。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

14. 【2020 统考真题】定义三元组 (a, b, c) (a, b, c 均为整数) 的距离 $D = |a - b| + |b - c| + |c - a|$ 。

给定 3 个非空整数集合 S_1 、 S_2 和 S_3 ，按升序分别存储在 3 个数组中。请设计一个尽可能高效的算法，计算并输出所有可能的三元组 (a, b, c) ($a \in S_1, b \in S_2, c \in S_3$) 中的最小距离。例如 $S_1 = \{-1, 0, 9\}$ ， $S_2 = \{-25, -10, 10, 11\}$ ， $S_3 = \{2, 9, 17, 30, 41\}$ ，则最小距离为 2，相应的三元组为 $(9, 10, 9)$ 。要求：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 语言或 C++ 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

15. 【2025 统考真题】有两个长度均为 n 的一维整型数组 A 和 res ，对数组 A 中的每个元素 $A[i]$ ，计算 $A[i]$ 与 $A[j]$ ($0 \leq i \leq j \leq n-1$) 乘积的最大值，并将其保存到 $res[i]$ 中。例如，当 $A[] = \{1, 4, -9, 6\}$ 时，得到 $res[] = \{6, 24, 81, 36\}$ 。现给定数组 A ，设计一个时间和空间上尽可能高效的算法 $calMulMax$ ，求 res 中各元素的值。函数原型为 `void calMulMax(int A[], int res[], int n)`。要求如下：

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

2.2.4 答案与解析

一、单项选择题

01. A

顺序表不像链表那样要在结点中存放指针域，因此存储密度大，选项 A 正确。选项 B 和 C 是链表的优点。选项 D 是错误的，比如对于树形结构，顺序表显然不如链表表示起来方便。

02. C

顺序表是顺序存储的线性表，表中所有元素的类型必须相同，且必须连续存放。一维数组中的元素可以不连续存放；此外，栈、队列和树等逻辑结构也可利用一维数组表示，但它与顺序表不属于相同的逻辑结构。在顺序表中，逻辑上相邻的元素物理位置上也相邻。

03. C

随机存取是指任在 $O(1)$ 的时间访问下标为 i 的元素，所需时间与顺序表中的元素个数 n 无关。

04. B

顺序表所占的存储空间 = 表长 $\times$ sizeof (元素的类型)，表长和元素的类型显然会影响存储空间的大小。若元素为结构体类型，则元素中各字段的类型也会影响存储空间的大小。

05. D

题干实际要求能最快存取第 $i-1$ 、 i 和 $i+1$ 个元素值。选项 A、B、C 都只能从头结点依次顺序查找，时间复杂度为 $O(n)$ ；只有顺序表可以按序号随机存取，时间复杂度为 $O(1)$ 。

06. A

只有顺序表可以按序号随机存取，且在最后进行插入和删除操作时不需要移动任何元素。

07. C

对说法 I，解析略；对说法 II，在最后位置插入新结点不需要移动元素，时间复杂度为 $O(1)$ ；对说法 III，被删结点后的结点需要依次前移，时间复杂度为 $O(n)$ ；对说法 IV，需要后移 $n-i$ 个结点，时间复杂度为 $O(n)$ 。

08. C

对说法 II，顺序表只需 3 次交换操作；链表需要分别找到两个结点前驱，第 4 个结点断链后再插入到第 2 个结点后，效率较低。对说法 III，需要依次顺序访问每个元素，时间复杂度相同。

09. C

需要将元素 $a_{i+1} \sim a_n$ 依次前移一位，共移动 $n-(i+1)+1=n-i$ 个元素。

10. C

在第 i 个位置插入一个元素，需要移动 $n-i+1$ 个元素，时间复杂度为 $O(n)$ 。

11. C

对 n 个元素进行排序的时间复杂度最小也要 $O(n)$ （初始有序时），通常为 $O(n\log_2 n)$ 或 $O(n^2)$ ，通过第 8 章学习后会更容易理解。选项 B 和 D 显然错误。顺序表支持按序号的随机存取方式。

12. D

顺序存储需要连续的存储空间，在申请时需申请 $n+m$ 个连续的存储空间，然后将线性表原来的 n 个元素复制到新申请的 $n+m$ 个连续的存储空间的前 n 个单元。

13. D

对于顺序存储的有序表，查找指定值元素可以采用顺序查找法或折半查找法，平均时间复杂度最少为 $O(\log_2 n)$ 。插入指定值元素需要先找到插入位置，然后将该位置及之后的元素依次后移一个位置，最后将指定值元素插入到该位置，平均时间复杂度为 $O(n)$ 。删除第 i 个元素需要将该元素之后的全部元素依次前移一个位置，平均时间复杂度为 $O(n)$ 。获取第 i 个元素只需直接根据下标读取对应的数组元素即可，时间复杂度为 $O(1)$ 。

二、综合应用题

01. 【解答】

算法思想：搜索整个顺序表，查找最小值元素并记住其位置，搜索结束后用最后一个元素填补空出的原最小值元素的位置。

本题代码如下：

```
bool Del_Min(SqList &L, ElemType &value) {
    //删除顺序表 L 中最小值元素结点，并通过引用型参数 value 返回其值
    //若删除成功，则返回 true；否则返回 false
    if (L.length == 0)
        return false; //表空，中止操作返回
    value = L.data[0];
    int pos = 0; //假定 0 号元素的值最小
    for (int i = 1; i < L.length; i++) //循环，寻找具有最小值的元素
        if (L.data[i] < value) { //让 value 记忆当前具有最小值的元素
            value = L.data[i];
        }
}
```

需要指向表尾结点，从而形成一个完整的环形结构，如图2.13所示。

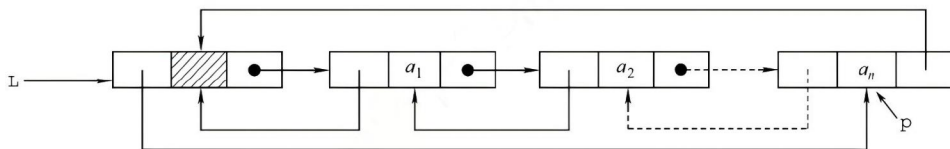


图 2.13 循环双链表

设尾结点为 *p，则 $p \rightarrow next$ 应指向头结点 L；同时， $L \rightarrow prior$ 应指向 *p。当循环双链表为空时，其头结点 L 的 prior 和 next 都指向自身 ($L \rightarrow prior == L$ 且 $L \rightarrow next == L$)。

2.3.5 静态链表

静态链表是用数组来模拟线性表的链式存储结构。每个结点都包含两个域：data 域和 next 域。与动态链表不同的是，这里的指针实际上是结点在数组中的相对地址（数组下标），也称游标。类似于顺序表，静态链表也需要预先分配一块连续的内存空间。

静态链表和单链表的对应关系如图 2.14 所示。

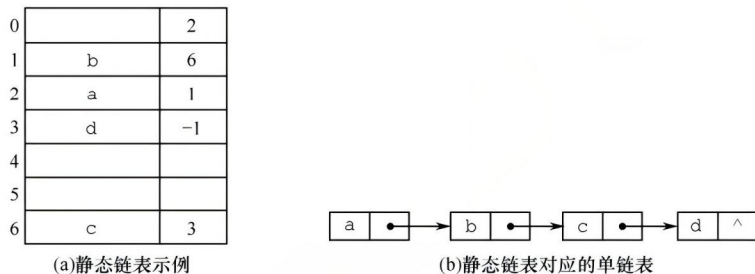


图 2.14 静态链表和单链表的对应关系

静态链表的结构类型定义如下：

```
#define MaxSize 50           //静态链表的最大长度
typedef struct {              //静态链表的结构类型定义
    ElemType data;           //存储数据元素
    int next;                 //下一个元素的数组下标
} SLinkList[MaxSize];
```

静态链表以 $next == -1$ 作为其结束标志。静态链表的插入、删除操作与动态链表类似，只需修改指针（数组下标），而无须移动元素。尽管静态链表在灵活性上不及动态链表，但在一些不支持指针的编程语言（如 Basic）中，它提供了一种巧妙的设计方案。

2.3.6 顺序表和链表的比较

1. 存取（读/写）方式

顺序表支持随机存取，可通过下标直接访问任意位置元素，时间复杂度为 $O(1)$ ，同时也支持顺序存取。链表仅支持顺序存取，必须从头结点开始逐个遍历。例如，访问第 i 个元素时，顺序表只需一次操作；而链表需要遍历 i 个结点，平均时间复杂度为 $O(n)$ 。

2. 逻辑结构与物理结构

顺序存储中，逻辑上相邻的元素在物理内存中也连续存放，其邻接关系由地址自然体现；链式存储中，逻辑相邻的元素在物理上未必相邻，其逻辑关系通过指针显式维护。

3. 查找、插入和删除操作

对于按值查找，若表无序，则两者的时间复杂度均为 $O(n)$ ；若表有序，则顺序表可以采用折半查找，时间复杂度为 $O(\log_2 n)$ 。对于按序号查找，顺序表的时间复杂度为 $O(1)$ ，链表则为 $O(n)$ 。对于插入/删除操作，顺序表平均需移动约一半的元素，开销较大；链表只需修改指针，无须移动元素，但前提是已知操作位置（否则仍需 $O(n)$ 的时间定位）。

4. 空间分配

顺序表在静态分配下需预先设定容量：过大造成内存浪费，过小则易在插入时溢出。动态分配虽支持运行时扩容，但需要申请新的连续内存块并复制全部原有数据，不仅耗时，还可能因系统缺乏足够连续空闲空间而失败。相比之下，链表采用动态结点分配，按需扩展，灵活性高，但每个结点需额外存储指针域，导致存储密度小于 1，空间利用率较低。

在实际应用中，该如何选择合适的存储结构呢？

1. 基于存储的考虑

当线性表的长度或规模难以预估时，顺序表因需预先分配固定容量而不适用；而链表无须预设容量，可按需动态扩展，但每个结点需额外存储指针，带来一定的空间开销。

2. 基于运算的考虑

若应用中频繁进行按序号访问（随机访问），顺序表具有 $O(1)$ 的优势，明显更高效；而对于以插入和删除为主的操作，链表在定位目标位置后仅需调整指针，开销较小。尽管定位过程本身仍需 $O(n)$ 时间，但在增删密集的场景下，链表的整体性能通常更优。

3. 基于环境的考虑

顺序表基于数组实现，几乎所有高级语言都支持，实现简单；链表则依赖指针操作，在部分环境中实现较为复杂。因此，开发语言和运行环境也是重要的考量因素。

综上所述，两种结构各有优劣。对于规模稳定、以随机访问为主的场景，宜采用顺序存储；而动态性强、频繁进行插入和删除操作的场景，则更适合链式存储。

注意

只有熟练掌握线性表的顺序存储和链式存储，才能深刻理解它们的优缺点。

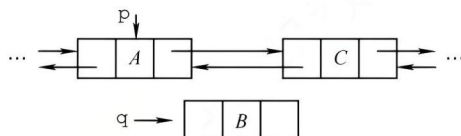
2.3.7 本节试题精选

一、单项选择题

01. 下列关于线性表的存储结构的描述中，正确的是（ ）。
- I. 线性表的顺序存储结构优于其链式存储结构
 - II. 链式存储结构比顺序存储结构能更方便地表示各种逻辑结构
 - III. 若频繁使用插入和删除结点操作，则顺序存储结构更优于链式存储结构
 - IV. 顺序存储结构和链式存储结构都可以进行顺序存取
- A. I、II、III B. II、IV C. II、III D. III、IV
02. 对于一个线性表，既要求能进行较快速地插入和删除，又要求存储结构能反映数据之间的逻辑关系，则应该用（ ）。
- A. 顺序存储方式 B. 链式存储方式 C. 散列存储方式 D. 以上均可以
03. 链式存储设计时，结点内的存储单元地址（ ）。

- A. 一定连续
B. 一定不连续
C. 不一定连续
D. 部分连续, 部分不连续
04. 下列关于线性表的说法中, 正确的是 ()。
- I. 顺序存储方式只能用于存储线性结构
II. 在一个设有头指针和尾指针的单链表中, 删除表尾元素的时间复杂度与表长无关
III. 带头结点的循环单链表中不存在空指针
IV. 在一个长度为 n 的有序单链表中插入一个新结点并仍保持有序的时间复杂度为 $O(n)$
V. 若用单链表来表示队列, 则应该选用带尾指针的循环链表
- A. I、II
B. I、III、IV、V
C. IV、V
D. III、IV、V
05. 设线性表中有 $2n$ 个元素, () 在单链表上实现要比在顺序表上实现效率更高。
- A. 删除所有值为 x 的元素
B. 在最后一个元素的后面插入一个新元素
C. 顺序输出前 k 个元素
D. 交换第 i 个元素和第 $2n-i-1$ 个元素的值 ($i=0, \dots, n-1$)
06. 在一个单链表中, 已知 q 所指结点是 p 所指结点的前驱结点, 若在 q 和 p 之间插入结点 s , 则执行 ()。
- A. $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
B. $p \rightarrow \text{next} = s \rightarrow \text{next}; s \rightarrow \text{next} = p;$
C. $q \rightarrow \text{next} = s; s \rightarrow \text{next} = p;$
D. $p \rightarrow \text{next} = s; s \rightarrow \text{next} = q;$
07. 给定有 n 个元素的一维数组, 建立一个有序单链表的最低时间复杂度是 ()。
- A. $O(1)$
B. $O(n)$
C. $O(n^2)$
D. $O(n \log_2 n)$
08. 将长度为 n 的单链表链接在长度为 m 的单链表后面, 其算法的时间复杂度采用大 O 形式表示应该是 ()。
- A. $O(1)$
B. $O(n)$
C. $O(m)$
D. $O(n+m)$
09. 单链表中, 增加一个头结点的目的是 ()。
- A. 使单链表至少有一个结点
B. 标识表结点中首结点的位置
C. 方便运算的实现
D. 说明单链表是线性表的链式存储
10. 在一个长度为 n 的带头结点的单链表 h 上, 设有尾指针 r , 则执行 () 操作与链表的表长有关。
- A. 删除单链表中的第一个元素
B. 删除单链表中的最后一个元素
C. 在单链表第一个元素前插入一个新元素
D. 在单链表最后一个元素后插入一个新元素
11. 对于一个头指针为 $head$ 的带头结点的单链表, 判定该表为空表的条件是 (); 对于不带头结点的单链表, 判定空表的条件为 ()。
- A. $head == \text{NULL}$
B. $head \rightarrow \text{next} == \text{NULL}$
C. $head \rightarrow \text{next} == head$
D. $head != \text{NULL}$
12. 在线性表 $a_0, a_1, \dots, a_{100}$ 中, 删除元素 a_{50} 需要移动 () 个元素。
- A. 0
B. 50
C. 51
D. 0 或 50
13. 通过含有 $n (n > 1)$ 个元素的数组 a , 采用头插法建立单链表 L , 则 L 中的元素次序 ()。
- A. 与数组 a 的元素次序相同
B. 与数组 a 的元素次序相反

- C. 与数组 a 的元素次序无关 D. 以上都错误
14. 下面关于线性表的一些说法中, 正确的是 ()。
- A. 对一个设有头指针和尾指针的单链表执行删除最后一个元素的操作与链表长度无关
B. 线性表中每个元素都有一个直接前驱和一个直接后继
C. 为了方便插入和删除数据, 可以使用双链表存放数据
D. 取线性表第 i 个元素的时间与 i 的大小有关
15. 在双链表中向 p 所指的结点之前插入一个结点 q 的操作为 ()。
- A. $p \rightarrow \text{prior} = q; q \rightarrow \text{next} = p; p \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{prior} = p \rightarrow \text{prior};$
B. $q \rightarrow \text{prior} = p \rightarrow \text{prior}; p \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{next} = p; p \rightarrow \text{prior} = q \rightarrow \text{next};$
C. $q \rightarrow \text{next} = p; p \rightarrow \text{next} = q; q \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{next} = p;$
D. $p \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{next} = p; q \rightarrow \text{prior} = p \rightarrow \text{prior}; p \rightarrow \text{prior} = q;$
16. 在双链表存储结构中, 删除 p 所指的结点时必须修改指针 ()。
- A. $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior};$
B. $p \rightarrow \text{prior} = p \rightarrow \text{prior} \rightarrow \text{prior}; p \rightarrow \text{prior} \rightarrow \text{next} = p;$
C. $p \rightarrow \text{next} \rightarrow \text{prior} = p; p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next};$
D. $p \rightarrow \text{next} = p \rightarrow \text{prior} \rightarrow \text{prior}; p \rightarrow \text{prior} = p \rightarrow \text{next} \rightarrow \text{next};$
17. 在如下图所示的双链表中, 已知指针 p 指向结点 A , 若要在结点 A 和 C 之间插入指针 q 所指的结点 B , 则依次执行的语句序列可以是 ()。



- ① $q \rightarrow \text{next} = p \rightarrow \text{next};$ ② $q \rightarrow \text{prior} = p;$ ③ $p \rightarrow \text{next} = q;$ ④ $p \rightarrow \text{next} \rightarrow \text{prior} = q;$
- A. ①②④③ B. ④③②① C. ③④①② D. ①③④②
18. 在双链表的两个结点之间插入一个新结点, 需要修改 () 个指针域。
- A. 1 B. 3 C. 4 D. 2
19. 在长度为 n 的有序单链表中插入一个新结点, 并仍然保持有序的时间复杂度是 ()。
- A. $O(1)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log_2 n)$
20. 与单链表相比, 双链表的优点之一是 ()。
- A. 插入、删除操作更方便 B. 可以进行随机访问
C. 可以省略表头指针或表尾指针 D. 访问前后相邻结点更灵活
21. 对于一个带头结点的循环单链表 L , 判断该表为空表的条件是 ()。
- A. 头结点的指针域为空 B. L 的值为 NULL
C. 头结点的指针域与 L 的值相等 D. 头结点的指针域与 L 的地址相等
22. 对于一个带头结点的循环双链表 L , 判断该表为空表的条件是 ()。
- A. $L \rightarrow \text{prior} == L \& \& L \rightarrow \text{next} == \text{NULL}$ B. $L \rightarrow \text{prior} == \text{NULL} \& \& L \rightarrow \text{next} == \text{NULL}$
C. $L \rightarrow \text{prior} == \text{NULL} \& \& L \rightarrow \text{next} == L$ D. $L \rightarrow \text{prior} == L \& \& L \rightarrow \text{next} == L$
23. 一个链表最常用的操作是在末尾插入结点和删除结点, 则选用 () 最节省时间。
- A. 带头结点的循环双链表 B. 循环单链表
C. 带尾指针的循环单链表 D. 单链表
24. 设对 n ($n > 1$) 个元素的线性表的运算只有 4 种: 删除第一个元素; 删除最后一个元素; 在第一个元素之前插入新元素; 在最后一个元素之后插入新元素, 则最好使用 ()。

- A. 只有尾结点指针没有头结点指针的循环单链表
B. 只有尾结点指针没有头结点指针的非循环双链表
C. 只有头结点指针没有尾结点指针的循环双链表
D. 既有头结点指针又有尾结点指针的循环单链表
25. 有两个长度为 n 的循环单链表, 若要求两个循环单链表头尾相接的时间复杂度为 $O(1)$, 则对应两个循环单链表各设置一个指针, 分别指向 ()。
A. 各自的头结点
B. 各自的尾结点
C. 各自的首结点
D. 一个表的头结点, 另一个表的尾结点
26. 有一个长度为 n 的循环单链表, 若从表中删除首元结点的时间复杂度达到 $O(n)$, 则此时采用的循环单链表的结构可能是 ()。
A. 只有表头指针, 没有头结点
B. 只有表尾指针, 没有头结点
C. 只有表尾指针, 带头结点
D. 只有表头指针, 带头结点
27. 某线性表用带头结点的循环单链表存储, 头指针为 head, 当 $\text{head} \rightarrow \text{next} \rightarrow \text{next} == \text{head}$ 成立时, 线性表的长度可能是 ()。
A. 0
B. 1
C. 2
D. 可能为 0 或 1
28. 有两个长度都为 n 的双链表, 若以 h_1 为头指针的双链表是非循环的, 以 h_2 为头指针的双链表是循环的, 则下列叙述中正确的是 ()。
A. 对于双链表 h_1 , 删除首结点的时间复杂度是 $O(n)$
B. 对于双链表 h_2 , 删除首结点的时间复杂度是 $O(n)$
C. 对于双链表 h_1 , 删除尾结点的时间复杂度是 $O(1)$
D. 对于双链表 h_2 , 删除尾结点的时间复杂度是 $O(1)$
29. 一个链表最常用的操作是在最后一个元素后插入一个元素和删除第一个元素, 则选用 () 最节省时间。
A. 不带头结点的循环单链表
B. 双链表
C. 单链表
D. 不带头结点且有尾指针的循环单链表
30. 需要分配较大连续空间, 插入和删除不需要移动元素的线性表, 其存储结构为 ()。
A. 单链表
B. 静态链表
C. 顺序表
D. 双链表
31. 下列关于静态链表的说法中, 正确的是 ()。
I. 静态链表兼具顺序表和单链表的优点, 因此存取表中第 i 个元素的时间与 i 无关
II. 静态链表能容纳的最大元素个数在表定义时就确定了, 以后不能增加
III. 静态链表与动态链表在元素的插入、删除上类似, 不需要移动元素
IV. 相比动态链表, 静态链表可能浪费较多的存储空间
A. I、II、III
B. II、III、IV
C. I、III、IV
D. I、II、IV
32. 【2016 统考真题】已知一个带有表头结点的循环双链表 L, 结点结构为

prev	data	next
------	------	------

, 其中 prev 和 next 分别是指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点, 正确的语句序列是 ()。
A. $p \rightarrow \text{next} \rightarrow \text{prev} = p \rightarrow \text{prev}; p \rightarrow \text{prev} \rightarrow \text{next} = p \rightarrow \text{next}; \text{free}(p);$
B. $p \rightarrow \text{next} \rightarrow \text{prev} = p \rightarrow \text{next}; p \rightarrow \text{prev} \rightarrow \text{next} = p \rightarrow \text{next}; \text{free}(p);$
C. $p \rightarrow \text{next} \rightarrow \text{prev} = p \rightarrow \text{next}; p \rightarrow \text{prev} \rightarrow \text{next} = p \rightarrow \text{prev}; \text{free}(p);$
D. $p \rightarrow \text{next} \rightarrow \text{prev} = p \rightarrow \text{prev}; p \rightarrow \text{prev} \rightarrow \text{next} = p \rightarrow \text{next}; \text{free}(p);$
33. 【2016 统考真题】已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将 f 存放于 1014H 处并插入单链表, 若 f 在逻辑上位于 a 和 e 之间, 则 a、e、f 的“链接地址”依次是 ()。

- A. 1010H、1014H、1004H B. 1010H、1004H、1014H
C. 1014H、1010H、1004H D. 1014H、1004H、1010H

34. 【2021 统考真题】已知头指针 h 指向一个带头结点的非空循环单链表, 结点结构为

data	next
------	------

, 其中 next 是指向直接后继结点的指针, p 是尾指针, q 是临时指针。现要删除该链表的第一个元素, 正确的语句序列是 ()。

- A. `h->next=h->next->next; q=h->next; free(q);`
B. `q=h->next; h->next=h->next->next; free(q);`
C. `q=h->next; h->next=q->next; if (p!=q) p=h; free(q);`
D. `q=h->next; h->next=q->next; if (p==q) p=h; free(q);`

35. 【2023 统考真题】现有非空双链表 L, 其结点结构为

prev	data	next
------	------	------

, prev 是指向直接前驱结点的指针, next 是指向直接后继结点的指针。若要在 L 中指针 p 所指向的结点 (非尾结点) 之后插入指针 s 指向的新结点, 则在执行语句序列 “`s->next=p->next;`” 后, 下列语句序列中还需要执行的是 ()。

- A. `s->next->prev=p; s->prev=p;`
B. `p->next->prev=s; s->prev=p;`
C. `s->prev=s->next->prev; s->next->prev=s;`
D. `p->next->prev=s->prev; s->next->prev=p;`

36. 【2024 统考真题】已知带头结点的非空单链表 L 的头指针为 h, 结点结构为

data	next
------	------

, 其中 next 是指向直接后继结点的指针。现有指针 p 和 q, 若 p 指向 L 中非首且非尾的任意一个结点, 则执行语句序列 “`q=p->next; p->next=q->next; q->next=h->next; h->next=q;`” 的结果是 ()。

- A. 在 p 所指结点后插入 q 所指结点 B. 在 q 所指结点后插入 p 所指结点
C. 将 p 所指结点移至 L 的头结点之后 D. 将 q 所指结点移动到 L 的头结点之后

二、综合应用题

- 在带头结点的单链表 L 中, 删除所有值为 x 的结点, 并释放其空间, 假设值为 x 的结点不唯一, 试编写算法以实现上述操作。
- 试编写在带头结点的单链表 L 中删除一个最小值结点的高效算法 (假设该结点唯一)。
- 试编写算法将带头结点的单链表就地逆置, 所谓“就地”是指辅助空间复杂度为 $O(1)$ 。
- 设在一个带表头结点的单链表中, 所有结点的元素值无序, 试编写一个函数, 删除表中所有处于给定的两个值 (作为函数参数给出) 之间的元素 (若存在)。
- 给定两个单链表, 试分析找出两个链表的公共结点的思想 (不用写代码)。
- 设 $C = \{a_1, b_1, a_2, b_2, \dots, a_n, b_n\}$ 为线性表, 采用带头结点的单链表存放, 设计一个就地算法, 将其拆分为两个线性表, 使得 $A = \{a_1, a_2, \dots, a_n\}$, $B = \{b_1, b_2, \dots, b_n\}$ 。
- 在一个递增有序的单链表中, 存在重复的元素。设计算法删除重复的元素, 例如 (7, 10,

10, 21, 30, 42, 42, 42, 51, 70) 将变为 (7, 10, 21, 30, 42, 51, 70)。

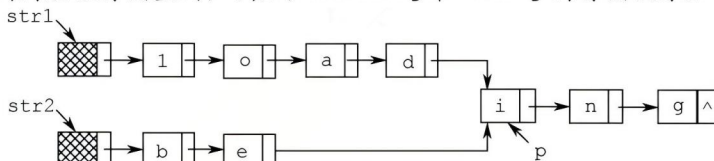
08. 设 A 和 B 是两个单链表 (带头结点), 其中元素递增有序。设计一个算法从 A 和 B 中的公共元素产生单链表 C , 要求不破坏 A 、 B 的结点。
09. 已知两个链表 A 和 B 分别表示两个集合, 其元素递增排列。编制函数, 求 A 与 B 的交集, 并存放于 A 链表中。
10. 两个整数序列 $A = a_1, a_2, a_3, \dots, a_m$ 和 $B = b_1, b_2, b_3, \dots, b_n$ 已经存入两个单链表中, 设计一个算法, 判断序列 B 是否是序列 A 的连续子序列。
11. 设计一个算法用于判断带头结点的循环双链表是否对称。
12. 有两个循环单链表, 链表头指针分别为 h_1 和 h_2 , 编写一个函数将链表 h_2 链接到链表 h_1 之后, 要求链接后的链表仍保持循环链表形式。
13. 设有一个带头结点的非循环双链表 L , 其每个结点中除有 pre 、 $data$ 和 $next$ 域外, 还有一个访问频度域 $freq$, 其值均初始化为零。每当在链表中进行一次 $Locate(L, x)$ 运算时, 令值为 x 的结点中 $freq$ 域的值增 1, 并使此链表中的结点保持按访问频度递减的顺序排列, 且最近访问的结点排在频度相同的结点之前, 以便使频繁访问的结点总是靠近表头。试编写符合上述要求的 $Locate(L, x)$ 函数, 返回找到结点的地址, 类型为指针型。
14. 设将 n ($n > 1$) 个整数存放到不带头结点的单链表 L 中, 设计算法将 L 中保存的序列循环右移 k ($0 < k < n$) 个位置。例如, 若 $k = 1$, 则将链表 $\{0, 1, 2, 3\}$ 变为 $\{3, 0, 1, 2\}$ 。要求:
 - 1) 给出算法的基本设计思想。
 - 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
 - 3) 说明你所设计算法的时间复杂度和空间复杂度。
15. 单链表有环, 是指单链表的最后一个结点的指针指向了链表中的某个结点 (通常单链表的最后一个结点的指针域是空的)。试编写算法判断单链表是否存在环。
 - 1) 给出算法的基本设计思想。
 - 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
 - 3) 说明你所设计算法的时间复杂度和空间复杂度。
16. 设有一个长度 n (n 为偶数) 的不带头结点的单链表, 且结点值都大于 0, 设计算法求这个单链表的最大孪生和。孪生和定义为一个结点值与其孪生结点值之和, 对于第 i 个结点 (从 0 开始), 其孪生结点为第 $n-i-1$ 个结点。要求:
 - 1) 给出算法的基本设计思想。
 - 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
 - 3) 说明你的算法的时间复杂度和空间复杂度。
17. 【2009 统考真题】已知一个带有表头结点的单链表, 结点结构为

data	link
------	------

假设该链表只给出了头指针 $list$ 。在不改变链表的前提下, 请设计一个尽可能高效的算法, 查找链表中倒数第 k 个位置上的结点 (k 为正整数)。若查找成功, 算法输出该结点的 $data$ 域的值, 并返回 1; 否则, 只返回 0。要求:

- 1) 描述算法的基本设计思想。
- 2) 描述算法的详细实现步骤。
- 3) 根据设计思想和实现步骤, 采用程序设计语言描述算法 (使用 C、C++ 或 Java 语言实现), 关键之处请给出简要注释。

18. 【2012 统考真题】假定采用带头结点的单链表保存单词, 当两个单词有相同的后缀时, 可共享相同的后缀存储空间, 例如, loading 和 being 的存储映像如下图所示。



设 str1 和 str2 分别指向两个单词所在单链表的头结点, 链表结点结构为

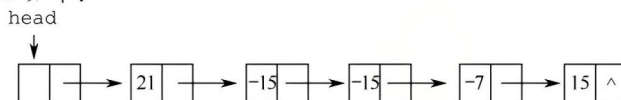
data	next
------	------

, 请设计一个时间上尽可能高效的算法, 找出由 str1 和 str2 所指向两个链表共同后缀的起始位置 (如图中字符 i 所在结点的位置 p)。要求:

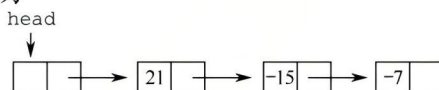
- 1) 给出算法的基本设计思想。
 - 2) 根据设计思想, 采用 C 或 C++ 或 Java 语言描述算法, 关键之处给出注释。
 - 3) 说明你所设计算法的时间复杂度。
19. 【2015 统考真题】用单链表保存 m 个整数, 结点的结构为

data	link
------	------

, 且 $|data| \leq n$ (n 为正整数)。现要求设计一个时间上尽可能高效的算法, 对于链表中 data 的绝对值相等的结点, 仅保留第一次出现的结点而删除其余绝对值相等的结点。例如, 若给定的单链表 head 如下:



则删除结点后的 head 为



要求:

- 1) 给出算法的基本设计思想。
 - 2) 使用 C 或 C++ 语言, 给出单链表结点的数据类型定义。
 - 3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
 - 4) 说明你所设计算法的时间复杂度和空间复杂度。
20. 【2019 统考真题】设线性表 $L = (a_1, a_2, a_3, \dots, a_{n-2}, a_{n-1}, a_n)$ 采用带头结点的单链表保存, 链表中的结点定义如下:

```

typedef struct node
{
    int data;
    struct node* next;
} NODE;
    
```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法, 重新排列 L 中的各结点, 得到线性表 $L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \dots)$ 。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- 3) 说明你所设计的算法的时间复杂度。

2.3.8 答案与解析

一、单项选择题

01. B

两种存储结构适用于不同的场合, 不能简单地谁说好谁坏, 说法 I 错误。链式存储用指针表

示逻辑结构,而指针的设置是任意的,因此比顺序存储结构能更方便地表示各种逻辑结构,说法 II 正确。在顺序存储中,插入和删除结点需要移动大量元素,效率较低,说法 III 的描述刚好相反。顺序存储结构既能随机存取又能顺序存取,而链式结构只能顺序存取,说法 IV 正确。

02. B

首先直接排除选项 A 和 D。散列存储通过散列函数映射到物理空间,不能反映数据之间的逻辑关系,排除选项 C。链式存储能方便地表示各种逻辑关系,且插入和删除操作的时间复杂度为 $O(1)$ 。

03. A

链式存储设计时,各个不同结点的存储空间可以不连续,但结点内的存储单元地址必须连续。

04. D

顺序存储方式同样适用于存储图和树,说法 I 错误。删除表尾结点时,必须从头开始找到表尾结点的前驱,其时间与表长有关,说法 II 错误。循环单链表中最后一个结点的指针不是 NULL,而是指向头结点,整个链表形成一个环,因此不存在空指针,说法 III 正确。有序单链表只能依次查找插入位置,时间复杂度为 $O(n)$,说法 IV 正确。队列需要在表头删除元素,表尾插入元素,采用带尾指针的循环链表较为方便,插入和删除的时间复杂度都为 $O(1)$,说法 V 正确。

05. A

对于选项 A,在单链表和顺序表上实现的时间复杂度都为 $O(n)$,但后者要移动很多元素,因此在单链表上实现效率更高。对于选项 B 和 D,顺序表的效率更高。C 无区别。

06. C

s 插入后, q 成为 s 的前驱,而 p 成为 s 的后继,选择选项 C。

注 意

可能有读者认为选项 C 中的两条语句交换后才正确。实际上,因为本题插入位置的前后结点都有指针指示(这与前面介绍的插入操作是不同的),所以选项 C 中的语句顺序并不会造成断链。在此提醒读者在学习过程中一定要多动脑思考,而不要生搬硬套。

07. D

若先建立链表,然后依次插入建立有序表,则每插入一个元素就需遍历链表寻找插入位置,即直接插入排序,时间复杂度为 $O(n^2)$ 。若先将数组排好序,然后建立链表,建立链表的时间复杂度为 $O(n)$,数组排序的最好时间复杂度为 $O(n\log_2 n)$,总时间复杂度为 $O(n\log_2 n)$ 。故选择选项 D。

08. C

先遍历长度为 m 的单链表,找到该单链表的尾结点,然后将其 next 域指向另一个单链表的首结点,其时间复杂度为 $O(m)$ 。

09. C

单链表设置头结点的目的是方便运算的实现,主要好处体现在:第一,有头结点后,插入和删除数据元素的算法就统一了,不再需要判断是否在第一个元素之前插入或删除第一个元素;第二,不论链表是否为空,其头指针是指向头结点的非空指针,链表的头指针不变,因此空表和非空表的处理也就统一了。

10. B

删除单链表的最后一个结点需置其前驱结点的指针域为 NULL,需要从头开始依次遍历找到该前驱结点,需要 $O(n)$ 的时间,与表长有关。其他操作均与表长无关,读者可自行模拟。

11. B, A

在带头结点的单链表中,头指针 head 指向头结点,头结点的 next 域指向第一个元素结点,head->next==NULL 表示该单链表为空。在不带头结点的单链表中,head 直接指向第一个元素结点,head==NULL 表示该单链表为空。